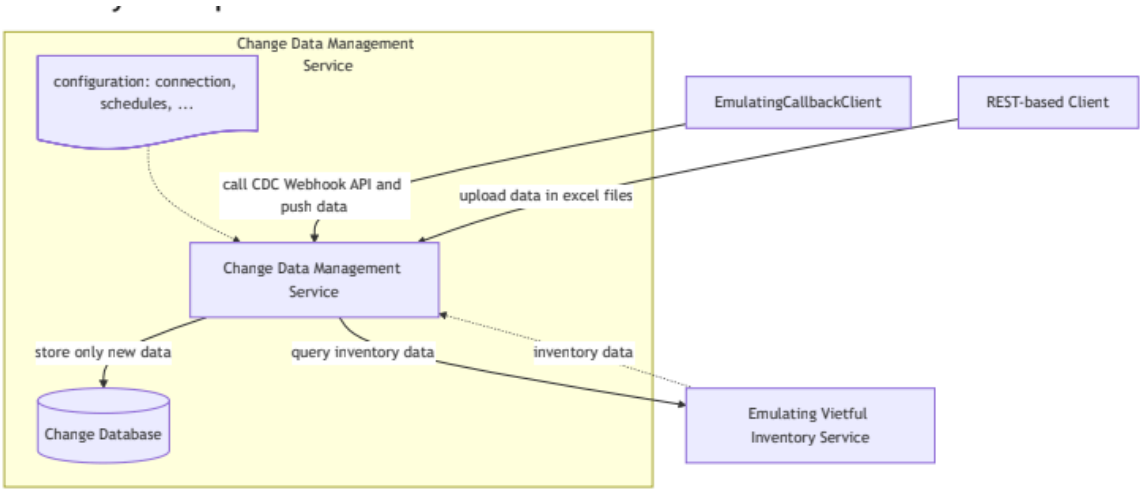
Change Data Management Service Project at kSynerX

I. Requirements

The goal of this assignment is to develop a prototype of a simple Change Data Management Service (CDMS) that stores only new data and data changes. The assignment includes:

- Emulating a real inventory service with limited functionality.
- Developing a change data service that supports different data ingestion mechanisms.
- Implementing exactly-once change processing with failure handling.
- Testing the system under spike loads.



II. Data Ingestion Mechanisms

1. Scheduled Polling

CDMS automatically retrieves product data from the inventory service at configurable intervals, for example, every five minutes.

The service:

- Calls the inventory API.
- Retrieves the product list.
- Compares incoming data with previously recorded data.
- Stores new products or changes to existing products and skips unchanged data.

2. Webhook

An external system calls the CDMS webhook API to push product data.

For example, when a product changes in the inventory system, the sender submits the updated product information to CDMS. CDMS receives the request and initiates processing.

3. Excel Upload

Clients upload Excel files through a REST API. CDMS processes the product data to add new products or update existing products.

III. Technologies and Scope

- **Emulated Vietful Inventory Service:** Based on the API documentation at Vietful API Documentation.
- **Database:** PostgreSQL.
- **CDMS:** A service supporting scheduled polling, webhook callbacks, and Excel uploads through a REST API.
- **Test Data Generation:** Faker is used to generate simulated data. See Faker Documentation.
- **Managed Data:** The focus is on inventory product data described in the Products API, including new products and changes to existing products.

IV. Key Challenges

1. Scheduled Data Retrieval

Configure polling intervals, handle pagination and timeouts, and prevent overlapping polling executions.

2. Concurrent Processing and Duplicate Prevention

Handle scheduled polling, Excel uploads, and webhook requests running concurrently without creating duplicate products, applying duplicate changes, or allowing older data to overwrite newer data.

3. Failure Recovery

Recover from failures in CDMS, the inventory service, the database, or the host machine.

This involves retry handling, polling/import progress tracking where appropriate, durable storage, and resuming unfinished work using an inbox and database transactions.

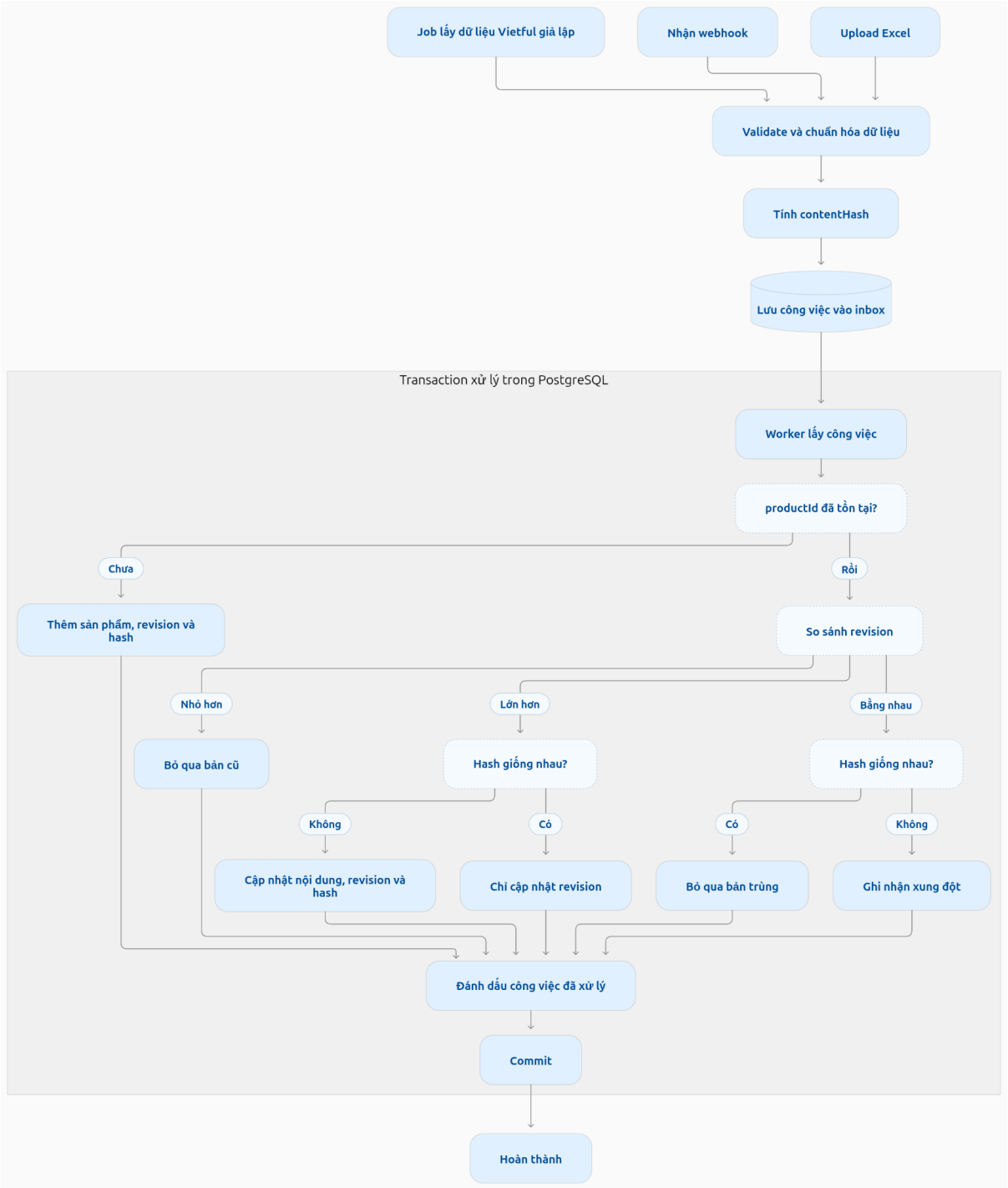
4. Data Version Ordering

Determine whether incoming product data is older than, identical to, or newer than the stored version.

The prototype uses a source-generated `revision` field to establish version order.

V. Processing Outcomes

Outcome	Next Action	Automatic Retry
`APPLIED`	Complete the event; the data has been applied.	No
`STALE`	Complete the event; the older version has been skipped.	No
`DUPLICATE`	Complete the event; the duplicate data has been skipped.	No
`CONFLICT`	Retain the event for investigation and conflict resolution.	Do not automatically retry the unchanged input.



VI. Implementation Status

Completed

- Scheduled polling.
- Webhook ingestion.
- A `revision` field has been added to the emulated source API to distinguish older and newer product versions.

Not Yet Completed

- Excel upload and ingestion.

Design Assumption

The original Product API response does not provide a revision or last-modified timestamp that establishes the order of updates. The prototype therefore extends the emulated source API with a `revision` field. This is a prototype-specific extension rather than a field provided by the original API.